

# Steam Data Engineering Nagyházi

## A választott architektúra

### Adatforrások

- [Kaggle adathalmaz](#): Steam API-n keresztül szerzett statikus adathalmaz, ami a játékok alapvető metaadatait tartalmazza. A Kaggle oldalon elérhető strukturált CSV és strukturálatlan JSON formátumú dataset is, először a CSV formátumút terveztem használni, de beolvasás után észrevettem, hogy több sorban el vannak csúszva az oszlopok, ezért végül a JSON formátumúval dolgoztam. A használt adathalmaznak 122611 sora és 42 oszlopa van.
- Steam API: A Steam API-ról lekértem az 500 legnépszerűbb játék (peak ccu szerint) átlagos achievement teljesítettség százalékát, és hozzáadtam a meglévő statisztikáimhoz (data enrichment). Azért döntöttem egy adat lekérdezése mellett a meglévők frissítése, vagy több adat hozzáadása helyett, mert nagy méretű az adathalmazom és a céljaimhoz megfelel egy adattal bővítenem a statisztikáimat, nagyobb művelet sokkal időigényesebb lett volna.

### Transzformáció

A strukturálatlan forrás batch feldolgozását a Python Pandas könyvtárral végeztem. Az eredeti adathalmazban kezeltem hiányzó értékeket. A 'publisher' és a 'developer' oszlopokban elsőt tartottam meg ha több volt, mert a sorok nagy részénél csak egy volt. Átalakítottam dátum formátumra a 'release\_date' oszlopot. Az indexet átneveztem game\_id-re, és átvettem indexből oszloppá, mert az indexként kezelt érték a Steam API-ban jelen lévő játékazonosító volt. A lekérdezett adatból kiszámítottam az achievement teljesítettség átlagos százalékát. A tisztított és kiszámított adatokat csillagsémába rendeztem dimenzió- és ténytáblák létrehozásával.

### Adattárolás

A letöltött adatokat lokálisan tároltam. A tisztított adatokat lokálisan elmentettem CSV formátumban, és feltöltöttem egy PostgreSQL relációs adatbázisba. Az adatbázis betöltésénél az idempotenciát a betöltésnél azzal biztosítom, hogy ha a betöltendő táblák már léteznek, azokat betöltés előtt törölöm kaszkádosítva, hogy ne sérüljenek függőségek.

### Orchestration

A pipelineok ütemezését, hibakezelését, naplózását a Prefect végzi. A kódomat először python notebook formátumban írtam, hogy könnyen futtatható legyen kisebb részenként az egyszerűbb hibakezelésért. A megírt notebookokat átírtam .py fájlalba, ahol a

Prefect könyvtárat használva külön választottam a pipeline elemeit task-okra és azokat flow-okba rendeztem. A flow-ok (Kaggle-t feldolgozó, API-t lekérdező, Postgres-t betöltő) a master pipelineon futnak egymás után. A fő flow minden nap UTC 00:00 (Budapest 02:00, az időzóna különbség miatt)-kor automatikusan lefut.

## Adatkiszolgálás

Az adatokat egy Metabase dashboard-ban jelenítem meg. A megjelenített adatok egy részéhez írtam SQL view lekérdezéseket, de olyan is van amit Metabasen belül készítettem. A dashboard 5 lekérdezés ábráját tartalmazza, és magyarázatot mellettük.

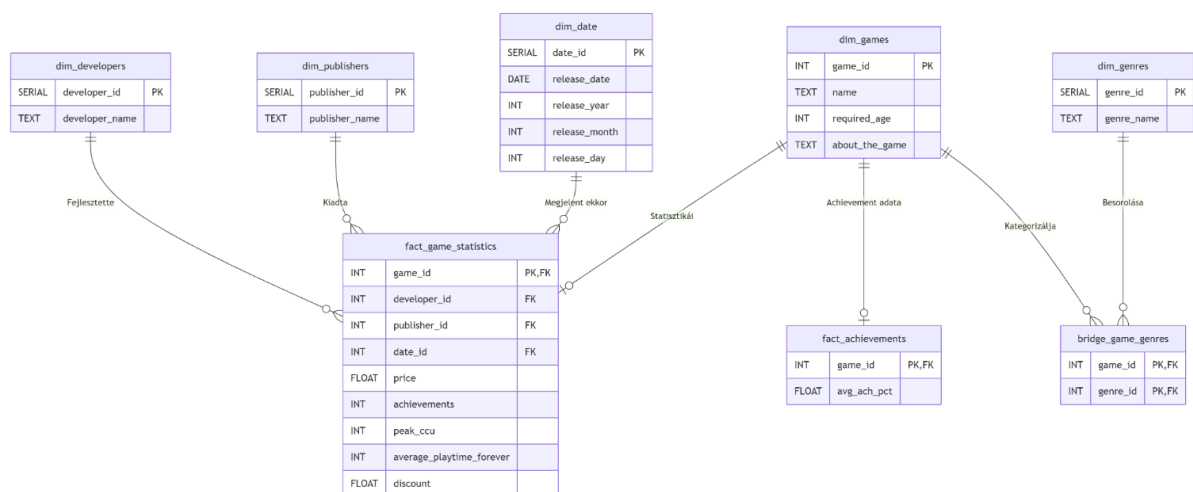
## Infrastruktúra

A reprodukálhatóság érdekében a projektet konténerizáltam, Docker Compose segítségével, így a pipeline lefuttatható idegen környezetből is.

## Adatmodell

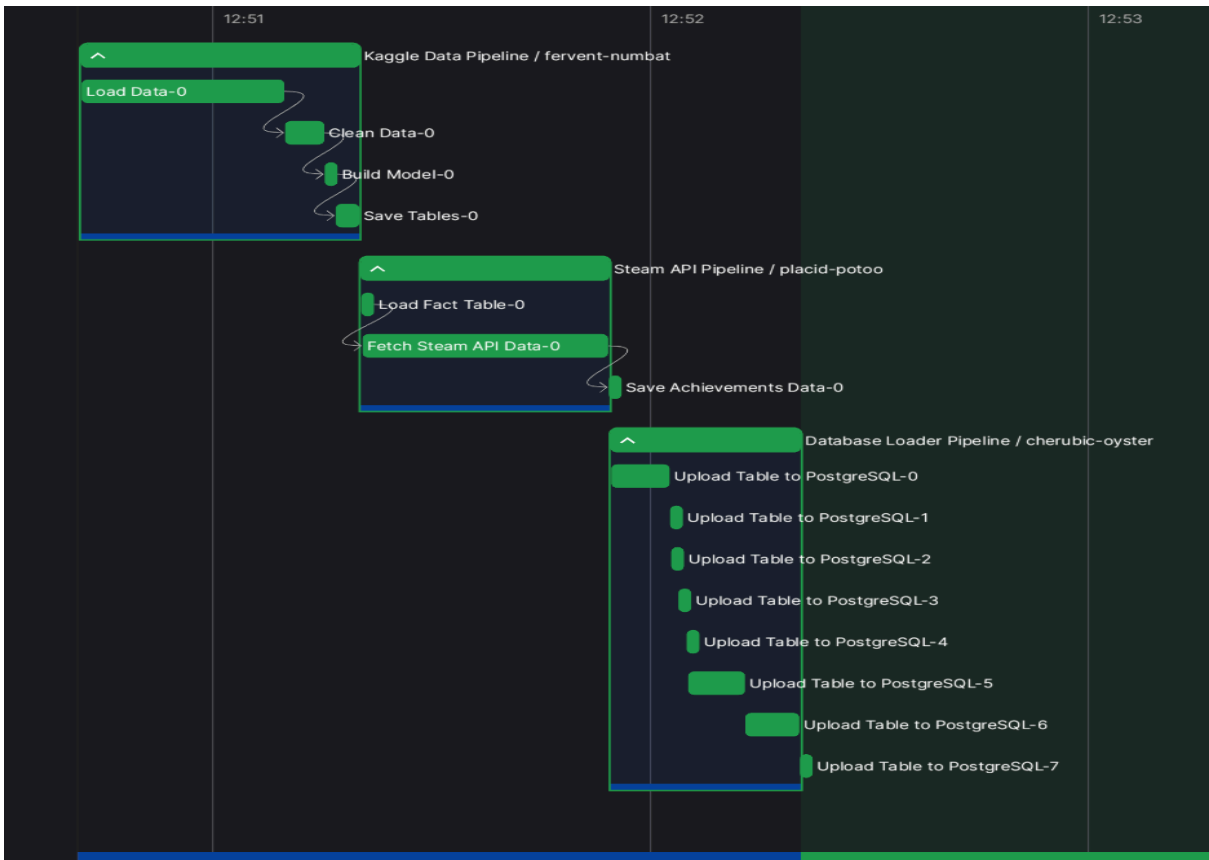
A feldolgozott adataimat csillag sémába rendeztem, hogy a lekérdezések teljesítményét javítsam. A csillagsémán a specifikációmhoz képest változtattam részben a kapott visszajelzés, részben az architektúra jobb megértése miatt. A modell egy központi tény táblából, egy azt kiegészítő API-ból szerzett tény táblából, hozzájuk tartozó dimenzió táblákból és egy híd-táblából áll. A híd táblára a műfajok és a játékok közötti N:N kapcsolat miatt volt szükség.

Az adatmodell felépítése:

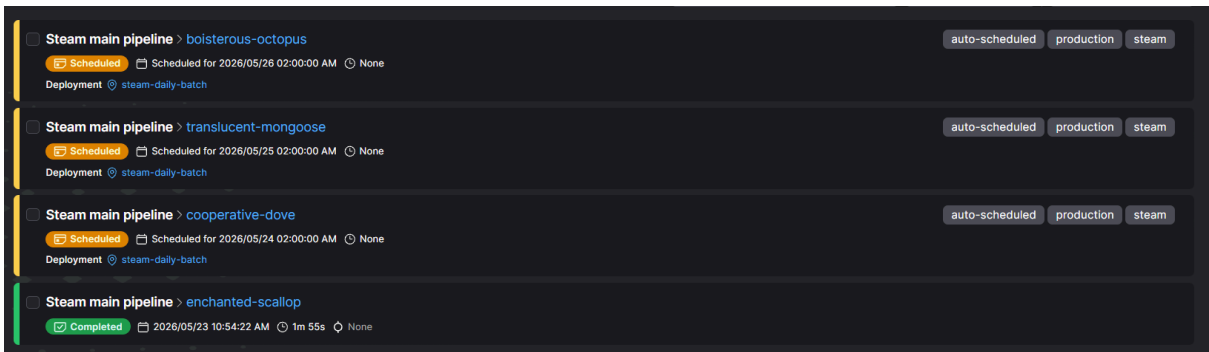


# Futtatás eredménye

Pipeline futtatása Prefectben:



Prefectben az is látszik, hogy a beütemezett futások mikor fognak elindulni:



A Metabase dashboardom 5 ábrát tartalmaz, ezek közül az egyik:

